CSCE 3201: Activity 1

Task 1: Problem Identification

1.Outcome Variable
The target variable for this analysis is **total_revenue**. This is a continuous numerical variable that represents the financial success of a given product.

2.Input Variables

The predictor variables used to provide information about the outcome variable include:

- **Numerical:** price, discount_percent, rating
- **Categorical:** product_category, customer_region

3.Learning Approach

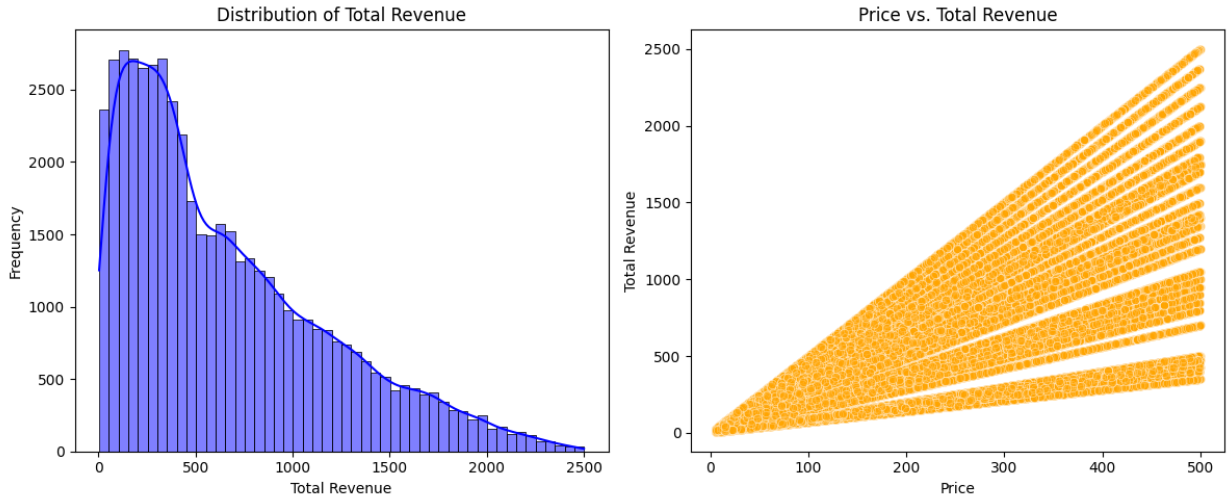
Because the target variable (total_revenue) is continuous, the appropriate supervised learning framework for this problem is **Regression**.

4.Selected Machine Learning Method

The algorithm chosen for this dataset is the **Random Forest Regressor**.
(Note: Because this project uses a Regression framework rather than Classification, regression-specific metrics (MAE, MSE, R-squared) were evaluated in lieu of a confusion matrix.)

Justification: The e-commerce data is often filled with non-linear and complicated relationships. As an example, a huge discount may lead to large sales quantity but low total money, and definite categories of products do not act in the same way as other products. Random Forest is very appropriate in this task due to its innate ability to deal with these non-linearities, its ability to deal with the non-linearity offered by the standard sales data and its demand of less exhaustive feature scaling than its linear counterparts. It is the best option to define the factors that have the largest impact on sales.

5.Data Visualizations



- **Distribution of Total Revenue (Left):** The histogram displays a strong right-skewed distribution. This is highly representative of real-world e-commerce behavior, where the vast majority of products generate lower, standard revenue, while a small handful of highly successful items create a long "tail" of maximum revenue (stretching up to 2,500).

- **Price vs. Total Revenue (Right):** The scatter plot reveals distinct, radiating diagonal lines. This visualizes the mathematical relationship where Total Revenue = Price × Quantity Sold. Because quantity is a discrete integer (1, 2, 3, etc.), each line represents a different quantity tier, demonstrating a direct linear relationship while emphasizing that price alone cannot perfectly predict revenue without underlying demand data.

Task 2: Supervised Machine Learning Implementation

Code Implementation
Below is the Python implementation used to load the dataset, build the preprocessing pipeline, and train the Random Forest model.

Python:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# 1. Load the Dataset
df = pd.read_csv('amazon_sales_dataset.csv')

# 2. Data Preprocessing Setup
features = ['price', 'discount_percent', 'rating', 'product_category', 'customer_region']
target = 'total_revenue'
df = df.dropna(subset=features + [target])

X = df[features]
y = df[target]

numeric_features = ['price', 'discount_percent', 'rating']
categorical_features = ['product_category', 'customer_region']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

# 3. Model Implementation Pipeline
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1))
])

# 4. Train-Test Split and Training
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)

# 5. Evaluation
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

Model Performance

- **Mean Absolute Error (MAE):** 280.17
- **Mean Squared Error (MSE):** 142647.39
- **R-squared (R2):** 0.4739

Performance Discussion & Improvement Strategies

The R-squared value of the Random Forest model is 0.4739, which indicates that the input features were able to explain an approximate of 47.4 percent of the total revenue. The MAE shows that the predictions of the model are erroneous by an average of about $280.17. Although this is a good starting point of a model that only established demand based on metadata and pricing, it can be optimised. The high MSE is directly due to the fact that regression algorithms measure the amount of error, which is of the order of magnitude of the error of the deposits that were of a high-revenue.

To enhance the betterment of such results, a subsequent iterations should be applied to maximize the tree depth and estimators number. Also, a prediction of the order date would allow the model to more deeply look into the context of order date critical moments such as holiday shopping, which would likely appear to the model as a higher level of predictive power.

Task 3: Reflection

The main problem that was observed as a result of this dataset analysis was the preprocessing pipeline of real-world e-commerce data set that entails continuous financial data and high-cardinality categorical metadata. Such characteristics as different product orthogonal segments and varied customer locations needed to be carefully combined to create a strong scikit-learn Pipeline and numerical scalers. To avoid data leakage in the process of training and testing, it was important to structure the entire process appropriately. In addition, the fact that e-commerce revenue is highly-right-skewed underscored why the choice of an ensemble algorithm rather than standard linear models would have been unable to cope with the e-commerce outliers.

This group project confirmed the importance of pipeline maintenance and data cleaning order. The Shift For a local environment The . For example, a scan A local environment is a programming setup that operates a dynamic scripting language in a local IDE environment. The coding in a modular, repeatable way with the help of ColumnTransformer not only made the modelling stage smoother; it laid a clear foundation, upon which it will become far more possible to deploy the model into a cloud service. Finally, it was a learning experience that the algorithmic selection represents but a single fraction of the puzzle, and it is how carefully the input features are realized before training that acts as the real bottleneck in going to produce actionable insights.